



Reinforcement Learning

Group Q

Diogo Carvalho - 20221935
Ricardo Pereira - 20250343
Yehor Malakhov - 20221691

June 05, 2026

Contents

1. Introduction	1
1.1. Sepsis	1
1.2. Reinforcement Learning for Sepsis Treatment	1
2. Algorithms	2
2.1. Metrics	2
2.2. Hyperparameter Tuning	2
2.3. Configuration A	2
2.3.1. Policy Iteration	3
2.3.1.1. Non Invertible Matrices	3
2.3.2. Q-Learning	4
2.3.2.1. Q-Learning Hyperparameter Tuning	4
2.3.3. Configuration A Results	5
2.4. Configuration B	5
2.4.1. Deep Q-Network (DQN)	6
2.4.1.1. Value Function and Training Objective	6
2.4.1.2. DQN Hyperparameter Tuning	6
2.4.2. Proximal Policy Optimisation (PPO)	7
2.4.2.1. PPO Hyperparameter Tuning	8
2.4.3. Configuration B Results	9
3. Conclusions	10
3.1. Final Results	10
3.2. Limitations and Future Work	10
4. Bibliography	11
5. Annexes	12

1. Introduction

1.1. Sepsis

Sepsis is a life-threatening clinical condition with extensive physiological and biochemical abnormalities. The Third International Consensus currently defines sepsis as “organ dysfunction caused by a dysregulated host response to infection”.^[1] Approximately 49 million people are affected by sepsis every year, and it is estimated that 11 million deaths are caused by the disease, accounting for up to 19.7% of all deaths worldwide.

Clinical understanding is continuously developing toward an immunological perspective, but it has not yet been possible to transform knowledge into evidence-based practice for the effective treatment of sepsis.

The use of machine learning methods to evaluate clinical data may help to identify further markers and patient subclasses which are associated with severity and outcome.

1.2. Reinforcement Learning for Sepsis Treatment

Reinforcement Learning (RL) is applied to sepsis as a sequential decision making problem, where an agent decides optimal treatment policies by observing patient states and selecting interventions such as IV fluid and vasopressor dosages to maximise long-term patient outcomes.

RL models can use historical ICU (Intensive Care Unit) data to learn personalised treatment strategies, with patient conditions represented as states and treatment decisions represented as actions.^{[2], [3]} Subsequent studies have refined these methods and reported promising results.

However, most existing approaches rely on discretized dosages because dosage distributions are highly uneven, which limits clinical usefulness, providing broad treatment ranges rather than precise dosages.

2. Algorithms

2.1. Metrics

We measure the performance of our algorithms by evaluating the average return and survival rate, the latter of which corresponds to the percentage of episodes where the return is greater than 0.

2.2. Hyperparameter Tuning

To achieve the best results possible, we performed hyperparameter tuning for all our algorithms using Optuna, an open-source library for hyperparameter optimisation.^[4] We defined a hyperparameter grid for each algorithm and ran multiple trials to find the best hyperparameters that maximise the average return. The details of the hyperparameter tuning process and the best hyperparameters found for each algorithm are described in the respective sections below.

2.3. Configuration A

Configuration A consists of a discrete environment with 716 states, corresponding to the clinical state of the patient, and 25 actions corresponding to 5 vasopressor levels and 5 IV-fluid dose levels. An episode ends when states 713 or 714 are reached. These states correspond to the death or survival of a patient, and give a reward of 0 or 1, respectively. The discount factor for this environment is 1, thus, we won't be mentioning it in the hyperparameter tuning sections, as it is not a hyperparameter that we can tune.

The environment is discrete and has a small state space, along with this, we were provided with the transition probabilities and reward function of the environment, which makes it possible to use model-based algorithms, in our case, policy iteration. Policy iteration can rapidly compute the optimal policy for this environment, given the transition probabilities and reward function, this provided us with a benchmark to compare our other configuration A algorithm against. Along with this, we implemented a model-free algorithm. When faced with the choice between Q-Learning and SARSA, we chose Q-Learning, as it is an off-policy algorithm. Off-policy algorithms learn the value of the optimal policy independently of the agent's actions, which makes more sense in our environment, as it would be like learning the best practice from different doctors' policies, instead of learning the best practice from a single doctor. In theory, given infinite exploration, both Q-Learning and SARSA would converge to the optimal policy. However, in practice, due to the limited number of episodes we can run, and the fact that we cannot guarantee infinite exploration, it is more likely that Q-Learning will converge to a better policy than SARSA, hence our choice.

The reason behind the choice of these algorithms is that since the environment is discrete and has a small state space, we can compute the optimal policy through tabular methods, along with this, we were provided with the transition probabilities and reward function, which makes it possible to use model-based algorithms.

Besides that, we tried to understand the environment by visualising the results of the random policy on episodes. In Figure 1, you can see how most of the states are as likely to be visited, and that the returns observed are quite low, which was expected.

2.3.1. Policy Iteration

Given a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ) , where S is the set of states, A is the set of actions, P is the transition probability function, R is the reward function, and γ is the discount factor, we want to find an optimal policy π that maximises the return. We can do this through policy iteration, which consists of two steps: policy evaluation and policy improvement. In policy evaluation, we compute the value function V_π for a given policy π . The value function gives us an estimate of how good it is to be in each state when following the policy. In policy improvement, we update the policy to be greedy with respect to the value function, that is, we choose the action that maximises the return based on the current value function. We repeat these two steps until convergence, that is, until the policy does not change anymore.

We can compute the value function for a given policy by solving the Bellman Expectation Equation for V_π , which is as follows:

$$V_\pi(s) = \sum_a \pi(s, a) \sum_{s'} p(s' | s, a) [R(s, a, s') + \gamma V_\pi(s')] \quad (1)$$

$$V_\pi(s) = \sum_a \pi(s, a) \sum_{s'} P(s, a, s') R(s, a) + \gamma \sum_a \pi(s, a) \sum_{s'} P(s, a, s') V(s') \quad (2)$$

$$V_\pi(s) = \sum_a \pi(s, a) R(s, a) + \gamma \sum_{s'} \sum_a \pi(s, a) P(s, a, s') V(s') \quad (3)$$

$$V_\pi(s) = r_\pi(s) + \gamma \sum_{s'} P_\pi(s, s') V(s') \quad (4)$$

Where $s \in S$ and $a \in A$.

We can now solve this by iterating over the set of states and updating the value function until convergence. However, we can also solve this as a linear system of equations.

For all states at once:

$$V_\pi = r_\pi + \gamma P_\pi V_\pi \quad (5)$$

$$V_\pi - \gamma P_\pi V_\pi = r_\pi \quad (6)$$

$$(I - \gamma P_\pi) V_\pi = r_\pi \quad (7)$$

$$A V_\pi = r_\pi \quad (8)$$

Then we could solve this as a $ax = b$ linear system, but iff A is invertible.

2.3.1.1. Non Invertible Matrices

A matrix M is non-invertible, also known as singular, if its kernel is non-trivial, that is:

$$\exists v \neq 0 : Mv = 0 \quad (9)$$

Row stochastic matrices, such as P_π , have the property that the sum of each row is 1, that is:

$$\sum_{s'} P_{\pi}(s, s') = 1, \forall s \in S \quad (10)$$

$$P_{\pi}1 = 1 \quad (11)$$

With this property in mind (Equation 11), and taking into account the definition of A, we can prove that, for $\gamma = 1$, $[1 \ 1 \ \dots \ 1]^T$ is part of the kernel of A:

$$A1 = (I - P_{\pi})1 = I1 - P_{\pi}1 = 1 - 1 = 0 \quad (12)$$

Therefore, no matter the policy, A will always be singular when $\gamma = 1$.

This is a problem, since for our environment, $\gamma = 1$, and thus we cannot compute V_{π} by solving the linear system of equations. However, we can use least squares to compute an approximate solution to the system, which is what we do in our implementation.

The average return obtained with this implementation, when tested on 50 000 different seeds, was 0.7868, with a survival rate of 82.5% (Figure 2).

As it is an approximation, this implementation will not reach the optimal policy. However, it converges extremely fast, that is, in 4 iterations. The results in Figure 2 show two clear groups of well-treated and poorly-treated patients. Note the preference of the policy for shorter episodes.

2.3.2. Q-Learning

Q-Learning is an off-policy, model-free algorithm that optimises the action-value function $Q(s, a)$, which represents the expected return of taking action a in state s .^[5] The update rule for Q-Learning is as follows:

$$Q(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (13)$$

2.3.2.1. Q-Learning Hyperparameter Tuning

In order to tune the hyperparameters of our Q-Learning implementation, we used Optuna to perform a search over the following hyperparameter grid:

HYPERPARAMETER	VALUE/RANGE	DOMAIN
Learning Rate	$10^{-5}, 0.01$	Log
Starting Exploration Rate	1.0	
Minimum Exploration Rate	0.01, 0.1	Log
Exploration Fraction	0, 1	Linear

Each Optuna trial ran the Q-Learning algorithm for 400 000 episodes, and evaluated the average return on 15 000 different seeds. The best hyperparameters found were the following:

HYPERPARAMETER	VALUE
Learning Rate	0.1848

HYPERPARAMETER	VALUE
Starting Exploration Rate	1.0
Minimum Exploration Rate	1.285×10^{-4}
Exploration Fraction	0.6713

With these hyperparameters, we obtained an average return of 0.7074, with a survival rate of 81% (Figure 3). The difference between the average return and the survival rate can be attributed to some episodes having a high length. Besides that, both the mean return value and the episode length converged after 200 000 episodes.

2.3.3. Configuration A Results

As can be seen in Figure 4, both algorithms achieved better mean return than the random policy, with Policy Iteration performing better than Q-Learning.

2.4. Configuration B

Configuration B consists of a 47-dimensional continuous environment. These 47 dimensions correspond to the 47 clinical variables used in “The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care” article by Komorowski et al.^[6] Along with this, certain random events were added to the environment. The events are the following:

- Occasional Monitor Malfunction, which corrupts the current episode’s with Gaussian noise.
- Occasional Missing Lab Values, which makes a subset of the 47 clinical variables unavailable for the remainder of the episode.
- Rare Sudden Patient Death, which causes the patient to die, regardless of the actions taken by the agent.

These events were added to make the environment more realistic, modelling real-life situations such as equipment calibration issues, lab analysis delays, and unexpected patient deterioration, as a side effect, these events also make the environment more challenging for the agent. Just like in Configuration A (p. 2), the discount factor is 1.

As this is a continuous environment, with a large state space, the algorithms we implemented in configuration A are not suitable for this environment. As such, we had to implement another two algorithms for this configuration. We implemented one value-based algorithm, Deep Q-Network (DQN), and one policy-based algorithm, Proximal Policy Optimisation (PPO). Q-Learning gave us good results in configuration A, so we decided to implement DQN, which is a Q-Learning extension for continuous environments. We also wanted to implement a policy-based algorithm, and PPO is a stable algorithm due to its clipping mechanism, which prevents large updates to the policy, and thus prevents the algorithm from diverging. This is particularly important in our environment, as it contains the aforementioned random events, which can make the learning process more unstable.

For this configuration, we implemented the following algorithms:

- Deep Q-Network (DQN)
- Proximal Policy Optimisation (PPO)

The algorithms were implemented using the Stable Baselines3 library. You can find the documentation for the DQN and PPO on the Stable Baselines3 Documentation website. ^{[7], [8]}

2.4.1. Deep Q-Network (DQN)

The Deep Q-Network (DQN) algorithm^[9], introduced by Mnih et al. (2015), extends classical Q-learning to high-dimensional, continuous state spaces by approximating the action-value function $Q(s, a)$ with a deep neural network, referred to as the Q-network. In the context of Configuration B, the agent receives as input a 47-dimensional state vector comprising the clinical variables defined by Komorowski et al. (2018), and must learn a policy that maps these observations to a discrete action space representing treatment decisions.

2.4.1.1. Value Function and Training Objective

The Q-network is parameterised by weights θ and is trained to minimise the mean squared Bellman error:

$$L(\theta) = E_{(s,a,r,s')} \sim D \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (14)$$

where γ is the discount factor, and θ are the weights of a periodically-updated target network. The use of a separate target network stabilises training by decoupling the targets from the rapidly updating online network, preventing the feedback loop that arises when the same network generates both predictions and targets.

2.4.1.2. DQN Hyperparameter Tuning

The hyperparameter grid used for tuning the DQN algorithm is the following:

HYPERPARAMETER	VALUE/RANGE	DOMAIN
Learning Rate	$10^{-5}, 0.01$	Log
Buffer Size	$10^4, 5 \times 10^5$	Linear
Learning Starts	$10^3, 10^4$	Linear
Batch Size	32, 64, 96, 128, 160, 192, 224, 256	Categorical
Train Frequency	1, 4, 8, 16	Categorical
Polyak Update Coefficient	$10^{-3}, 1$	Log
Target Update Interval	1, 10^3	Linear
Exploration Fraction	0.01, 0.5	Linear
Starting Exploration Rate	1.0	
Minimum Exploration Rate	0.01, 0.1	Linear

All the unmentioned hyperparameters were left with their default values, as defined in the Stable Baselines3 documentation.

Unlike in Q-Learning Hyperparameter Tuning (p. 4), Random Search was used instead of TPESampler. Each Optuna trial ran the DQN algorithm for 2 000 000 timesteps, which is

roughly equivalent to 2.000×10^5 episodes, and evaluated the average return every 500 000 timesteps on 1.000×10^3 different seeds. The best hyperparameters found were the following:

HYPERPARAMETER	VALUE
Learning Rate	7.238×10^{-5}
Buffer Size	295 800
Learning Starts	1274
Batch Size	32
Train Frequency	1
Polyak Update Coefficient	0.2042
Target Update Interval	216
Exploration Fraction	0.3152
Starting Exploration Rate	1.0
Minimum Exploration Rate	0.017 68

After tuning the hyperparameters, we trained another DQN algorithm with the best found hyperparameters for 2 000 000 timesteps, which ended up being 243 800 episodes, and evaluated the average return on 15 000 different seeds. This was done for two reasons: first, due to a coding mistake, the survival rate was not being returned, and thus, we only had the average return; this same mistake was also made during the hyperparameter tuning phase for Proximal Policy Optimisation. Second, we wanted to evaluate on more seeds to get a better estimate of the average return and survival rate.

With these hyperparameters, we obtained an average return of 0.6565, with a survival rate of 70.1% (Figure 5). Interestingly, DQN was not improving for the first 50 000 episodes, yet right after it showed an unexpected jump in mean return. Notably, the algorithm did not fully converge on the 243 800th episode, but we could not explore it further, because of computational limitations.

2.4.2. Proximal Policy Optimisation (PPO)

Proximal Policy Optimisation^[10] is an on-policy, model-free policy gradient algorithm that improves training stability by constraining how much the policy is permitted to change in a single update step. At each iteration, PPO collects a batch of experience under the current policy and then performs multiple gradient updates on that data but rather than maximising the raw policy gradient objective, which can cause destructively large parameter updates, it optimises a clipped surrogate objective that acts as a conservative lower bound on the expected return of the updated policy. The surrogate objective is defined as:

$$L^{\text{CLIP}}(\theta) = E_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)A_t)] \quad (15)$$

where

$$r_{t(\theta)} = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}}(a_t | s_t) \quad (16)$$

is the probability ratio between the new and old policy, A_t is the estimated advantage at timestep t , and ε is the clipping hyperparameter. The min operator ensures that when the advantage is positive, meaning the action was better than expected, the policy update is capped at a ratio of $1 + \varepsilon$, preventing the algorithm from over-exploiting a single favourable observation. Symmetrically, when the advantage is negative, the ratio is floored at $1 - \varepsilon$. This clipping mechanism removes the incentive to move r_t outside the interval $[1 - \varepsilon, 1 + \varepsilon]$, effectively keeping the updated policy within a trust region of the old policy and preventing the large, destabilising parameter updates that naive policy gradient methods are susceptible to.

2.4.2.1. PPO Hyperparameter Tuning

The hyperparameter grid used for tuning the PPO algorithm is the following:

HYPERPARAMETER	VALUE/RANGE	DOMAIN
Learning Rate	$10^{-5}, 0.01$	Log
Number of Steps	128, 256, 512, 1024, 2048	Categorical
Batch Size	4, 8, 16, 32, 64	Categorical
Number of Epochs	3, 10	Linear
GAE Lambda	0.85, 0.99	Linear
Clipping Range	0.1, 0.3	Linear
Entropy Coefficient	$10^{-4}, 0.1$	Log
Value Function Coefficient	0.1, 1	Linear
Max Gradient Norm	0.3, 4.0	Linear

All the unmentioned hyperparameters were left with their default values, as defined in the Stable Baselines3 documentation.

Just like in DQN Hyperparameter Tuning (p. 6), Random Search was used instead of TPE-Sampler. Each Optuna trial ran the PPO algorithm for 2 000 000 timesteps, which is roughly equivalent to 2.000×10^5 episodes, and evaluated the average return every 500 000 timesteps on 1.000×10^3 different seeds. The best hyperparameters found were the following:

HYPERPARAMETER	VALUE
Learning Rate	1.099×10^{-4}
Number of Steps	512
Batch Size	16
Number of Epochs	3

HYPERPARAMETER	VALUE
GAE Lambda	0.9066
Clipping Range	0.2485
Entropy Coefficient	4.615×10^{-3}
Value Function Coefficient	0.3588
Max Gradient Norm	1.424

After tuning the hyperparameters, we trained another PPO algorithm with the best found hyperparameters for 2 000 000 timesteps, which ended up being 234 800 episodes, and evaluated the average return on 15 000 different seeds. The reasons for doing this were mentioned in the DQN Hyperparameter Tuning (p. 6) section.

With these hyperparameters, we obtained an average return of 0.6688, with a survival rate of 68.5% (Figure 6). Compared to DQN, PPO demonstrated faster learning, converging approximately after 70 000 episodes. Notably, this was close to the number of episodes required for DQN to begin learning. In other words, even though PPO was faster to learn, DQN understood the problem better.

2.4.3. Configuration B Results

As can be seen in Figure 7, both algorithms achieved better mean return than the random policy,

3. Conclusions

3.1. Final Results

	POLICY ITERATION	Q-LEARNING	DQN	PPO
	Configuration A		Configuration B	
Mean Return	0.7868	0.7074	0.6565	0.6688
STD Return Survivors	0.0474	0.1083	0.0485	0.0172
STD Return Non-Survivors	0.0434	0.0930	0.0447	0.0173
Survival Rate	82.5%	81%	70.1%	68.5%

The results between the two configurations are not directly comparable, as they are different environments; nonetheless, the results of configuration B being worse than the results of configuration A can be attributed to the fact that configuration B is a much more complex environment, with a much continuous state space instead of a discrete one, and thus it is expected that the algorithms would perform worse on it.

3.2. Limitations and Future Work

We believe that conducting only 30 hyperparameter search trials for DQN and PPO, that is, 30 unique hyperparameter combinations, was likely insufficient to identify the optimal hyperparameters for these algorithms. This limitation is particularly relevant given that the trials were conducted using Random Search rather than TPESampler. The decision to restrict the number of trials to 30 was driven by computational constraints, as the tuning process for these algorithms required a total of approximately 28 hours to complete. This is something that we could look to improve in future work, as it is likely that better hyperparameters could be found with a more extensive search, and maybe a better defined hyperparameter grid.

4. Bibliography

- [1] D. Jarczak, S. Kluge, and A. Nierhaus, “Sepsis—Pathophysiology and Therapeutic Concepts,” *Frontiers in Medicine*, 2021, doi: 10.3389/fmed.2021.628302[◦].
- [2] K. Choudhary, D. Gupta, and P. S. Thomas, “ICU-Sepsis: A Benchmark MDP Built from Real Medical Data,” in *Reinforcement Learning Conference*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.05646>[◦]
- [3] Y. Huang, R. Cao, and A. Rahmani, “Reinforcement Learning For Sepsis Treatment: A Continuous Action Space Solution,” in *Proceedings of the 7th Machine Learning for Healthcare Conference*, Z. Lipton, R. Ranganath, M. Sendak, M. Sjoding, and S. Yeung, Eds., in *Proceedings of Machine Learning Research*, vol. 182. PMLR, 2022, pp. 631–647. [Online]. Available: <https://proceedings.mlr.press/v182/huang22a.html>[◦]
- [4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, in KDD '19. Anchorage, AK, USA: Association for Computing Machinery, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701[◦].
- [5] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3, pp. 279–292, May 1992, doi: 10.1007/BF00992698[◦].
- [6] M. Komorowski, L. A. Celi, O. Badawi, A. C. Gordon, and A. A. Faisal, “The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care,” *Nature Medicine*, vol. 24, no. 11, pp. 1716–1720, Nov. 2018, doi: 10.1038/s41591-018-0213-5[◦].
- [7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, [Online]. Available: <https://jmlr.org/papers/v22/20-1364.html>[◦]
- [8] S. B. Team, “Stable-Baselines3 Documentation.” Accessed: June 01, 2026. [Online]. Available: <https://stable-baselines3.readthedocs.io/en/master/>[◦]
- [9] V. Mnih *et al.*, “Playing Atari with Deep Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1312.5602>[◦]
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms.” [Online]. Available: <https://arxiv.org/abs/1707.06347>[◦]

5. Annexes

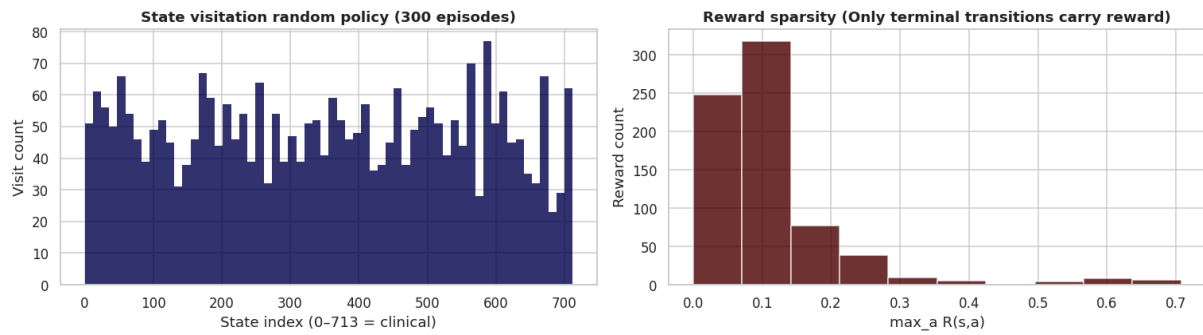


Figure 1: Environment Response to Random Policy

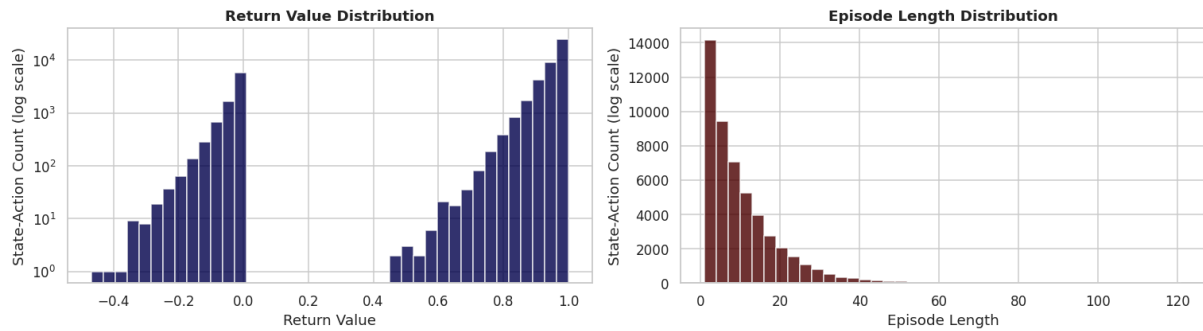


Figure 2: Policy Iteration Results

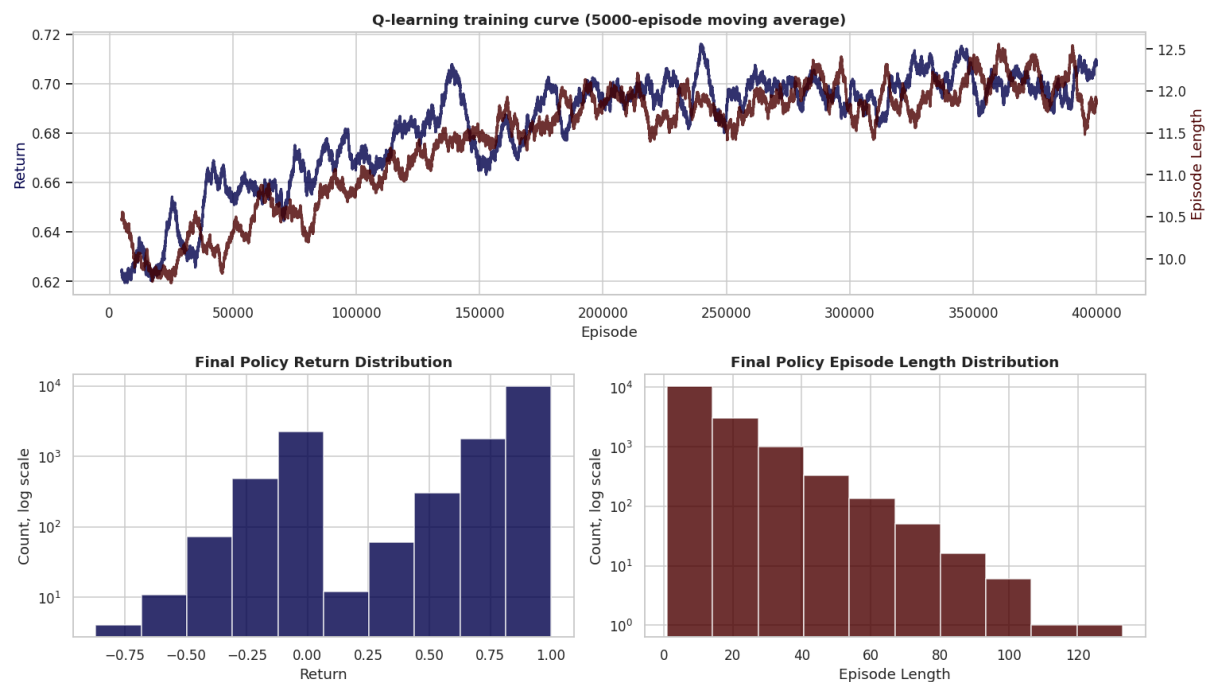


Figure 3: Q-Learning Performance Analysis

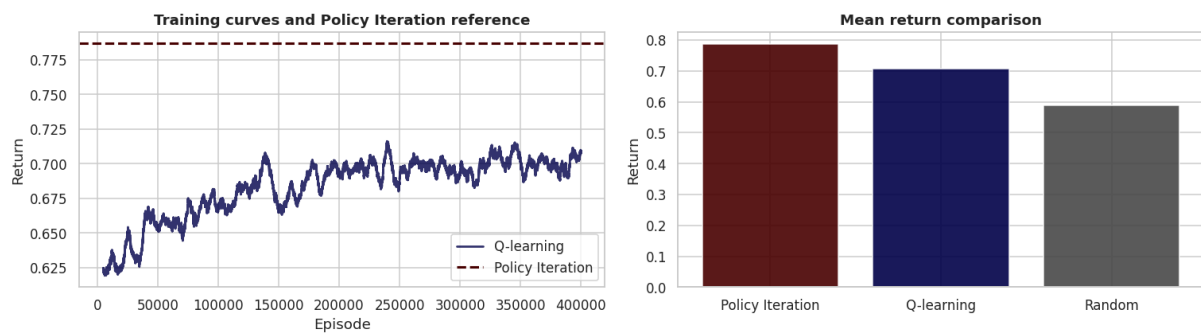


Figure 4: Configuration A Algorithm Comparison

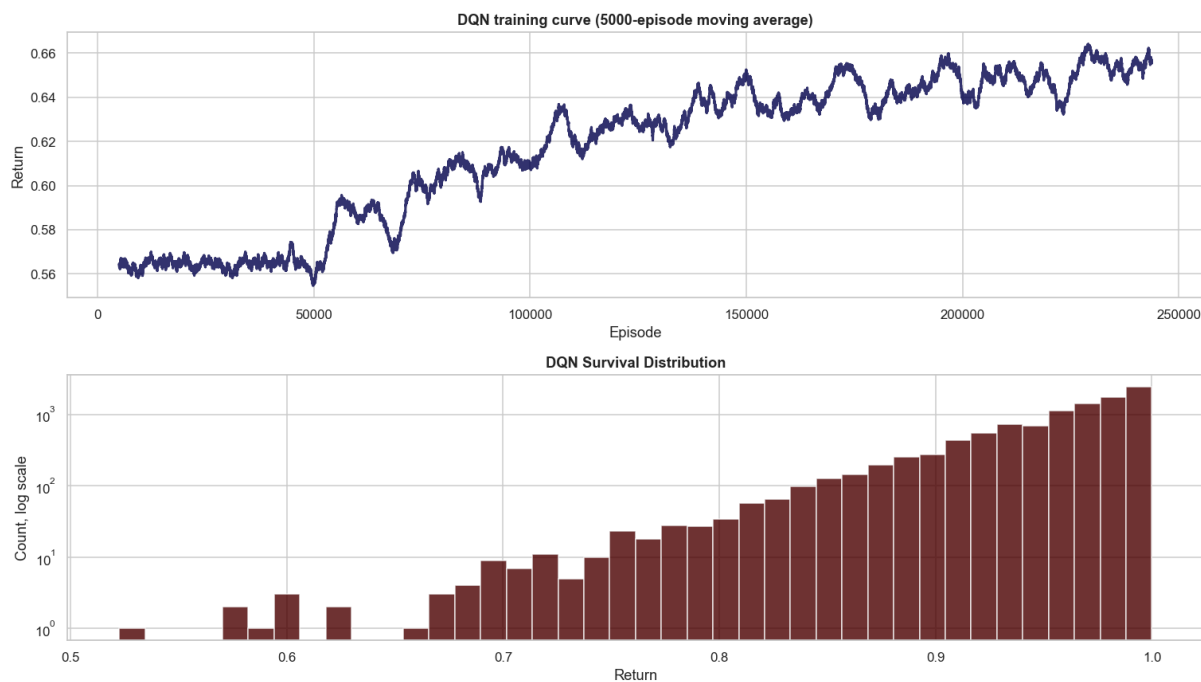


Figure 5: DQN Performance Analysis

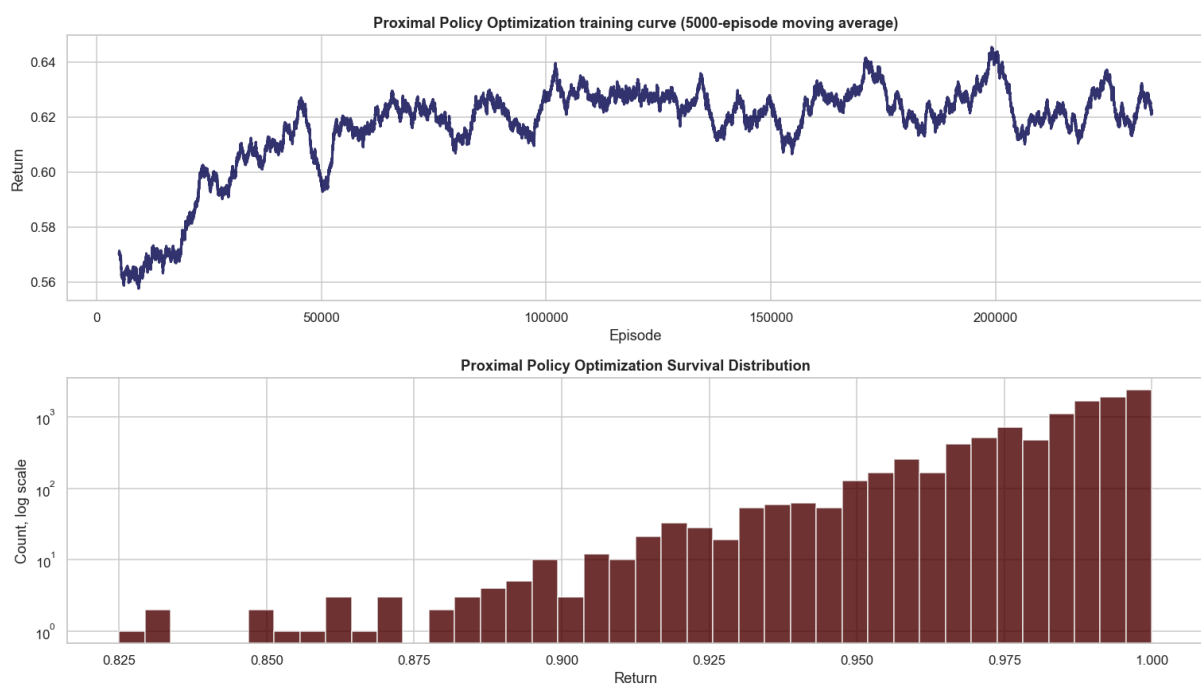


Figure 6: PPO Performance Analysis

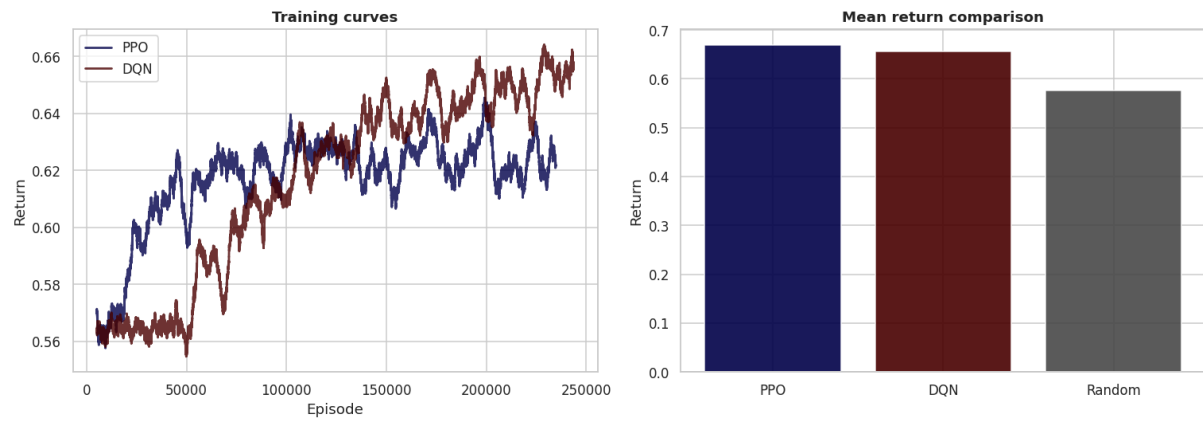


Figure 7: Configuration B Algorithm Comparison